Parity

Final project plan: SRS, strategy, free-credit plan and execution schedule

Verify that a migrated billing system behaves exactly like the legacy one, and show the evidence.

Item	Detail
Event	Meta Global AI Developer Hackathon (Cerebral Valley)
Track	Indie Developer. Theme: Push the Frontier (software)
Builder	Dev (solo)
Submission deadline	Nov 4, 2026, 9:00 PM Eastern (EST). That is 7:30 AM IST on Nov 5.
Target submission time	At least 90 minutes before the deadline
Primary models	Muse Spark Open (Meta open weights) through partner inference: Baseten first, Modal and Fireworks as backups
Dev machine	MacBook Air M4 (everything runs locally except model inference)

Read this first. The building period ends today. This plan is written to be executed in a compressed window, with a scaling table (Section 13) that tells you what to cut depending on how many hours you have left. A small project that runs end to end beats a large one that does not.

Contents

1 Summary and changes from the original pitch | 2 Winning strategy | 3 Scope | 4 System design | 5 Functional requirements | 6 Non-functional requirements | 7 Tech stack | 8 Models, providers and free credits | 9 Evaluation plan | 10 Report and Parity Score card | 11 File structure | 12 Prize playbook | 13 Execution plan | 14 Demo video script | 15 Submission checklist | 16 Risk register | 17 Definition of done | Appendix: verified facts and assumptions

1. Summary and changes from the original pitch

What Parity is. An autonomous agent that migrates legacy business logic (pricing and billing) and produces evidence that the new code behaves like the old code. It records what the original actually does on hundreds of nasty inputs, migrates the code, replays the same inputs, diagnoses every mismatch, repairs it, and finishes with a report: what was verified identical, what still differs, and which quirks in the old code look like bugs so a human can decide what to keep.

Wording rule for the demo and README. Say "verified identical on N recorded behaviors, including a held-out set the agent never saw." Do not say "proves". Differential testing is strong evidence, not formal proof, and technical judges will notice the difference.

Version 2 changes (why and what)

Change	Reason
Open-weights-first: Muse Spark Open via partner inference is the default engine	No Meta credit card needed. Directly targets the $10K Best Build on Open Weight Models prize. Still satisfies the Stage One requirement to use Meta's models.
Provider-agnostic client with automatic failover (Baseten, then Modal, then Fireworks)	Partner credits are not guaranteed. One outage or rate limit must not break the demo.
Muse Spark Open does migration and repair; Muse Code becomes optional	Muse Code's headless use and billing are unverified. Do not build the critical path on it.
New: red-team verifier agent	After convergence, a second agent reads both codebases and writes inputs designed to break equivalence. Strengthens technical depth, the held-out claim, and the Agentic prize case.
New: behavior contract	The recordings are turned into a short plain-English rule list, each rule backed by example inputs. The artifact a real team would keep.
New: Parity Score card	A shareable image/HTML card for the X post (Most Viral prize).
Cut: five sample modules reduced to three (1 development, 2 held-out); keep/fix decisions UI dropped	Time. Both are stretch items only.

2. Winning strategy

Judging rubric mapped to Parity

Criterion (weight)	What judges want	Parity's answer
Impact (30%)	Real pain, clear beneficiary, could become a real product	Legacy billing migrations fail silently (rounding, overflow, dates). Parity replaces "trust me" with a measured preservation rate and a bug list.
Use of Meta tooling (25%)	Creative use beyond a basic integration	Muse Spark Open as a multi-role agent (fuzzer, archaeologist, migrator, debugger, red team) with a usage ledger in every report, running on partner inference.
Technical depth (20%)	Sound engineering, real craft	Immutable hashed golden recordings, deterministic boundary fuzzing plus model-generated edge cases, no-regression repair rule, held-out and adversarial verification.
Demo (25%)	Works live, genuinely cool to watch	Live run on an unseen module: mismatch, diagnosis, patch, 100%. Then the report and the score card.

Prize targets and honest odds

Up to 20,000 builders were accepted, so the large placement prizes are a long shot for anyone. Parity's realistic edge is in the bonus awards, where fit matters more than polish.

Priority	Prize	Why Parity fits	What it needs from you
1	Best Build on Open Weight Models ($10K)	Core engine is Muse Spark Open on partner inference.	Default run uses only open-weight models. README names the provider and exact model ID. Keep dashboard usage screenshots.
2	Best Agentic Use-Case ($5K)	Multi-step autonomous loop with measurable success.	Show iterations done without human input, success rate, and the agent decision timeline.
3	Indie track top 5 (up to $150K)	Strong rubric fit.	Polished 3-minute demo and a clean repo.
4	Most Viral ($20K)	A scary, visual bug story is shareable.	Post the clip and score card on X right after submitting; submit the post link on the platform.
5	Community Vote ($10K)	One vote per verified account.	Share your gallery link with classmates and communities.

3. Scope

In scope (MVP)

- Legacy C modules for pricing and billing logic, migrated to Python 3.
- Why C to Python: clang ships with macOS, and the two languages differ in exactly the ways billing code breaks (32-bit signed overflow, integer division, rounding and float printing, calendar math).
- Pure functions only: numbers and strings in, numbers and strings out. No file, network, clock or random access in the modules under test.
- The full loop: input generation, recording, migration, replay, diagnosis, repair, held-out check, red-team check, quirk triage, report.

Out of scope

- Whole-application migration, GUIs, formal verification, additional languages, database or file side effects.
- Any use of code you do not have rights to. All sample legacy modules are written by you.

4. System design

Pipeline

#	Stage	What happens	Engine	Output
1	Archaeologist	Reads the legacy module, extracts function signatures, and notes suspicious logic worth probing.	Muse Spark Open	Signature list, probe notes
2	Fuzzer	Builds inputs: deterministic boundary seeds by type (zero, one, negatives, integer limits, rounding ties, leap days, month ends) plus model-written edge cases plus mutations.	Code + Muse Spark Open	inputs file (300+ per function)
3	Recorder	Compiles a driver and runs every input on the original under a timeout. Writes the golden file once and hashes it.	clang, subprocess	Immutable golden file + hash
4	Migrator	Translates the module to Python 3 with identical function names and signatures. First-pass result is saved as the baseline.	Muse Spark Open	Migrated module (baseline)
5	Replayer	Runs the same inputs on the migrated code and compares strictly.	Code	Mismatch set, match rate
6	Debugger	Shrinks failing inputs, clusters mismatches, diagnoses with a closed taxonomy, proposes a patch. A patch is accepted only if no previously matching input regresses. Max 6 rounds.	Muse Spark Open	Patched module, history
7	Held-out check	Fresh seed and fresh model-generated inputs replayed against the final code.	Code + Muse Spark Open	Held-out match rate
8	Red team	Reads legacy and migrated code and writes inputs designed to break equivalence. Two rounds. Any break goes back to the Debugger.	Muse Spark Open	Adversarial results
9	Quirk triage and behavior contract	Labels suspicious preserved behaviors (likely bug, likely intentional, unclear) with evidence. Writes the plain-English rule list.	Muse Spark Open	Quirk table, contract
10	Reporter	Builds the HTML/JSON report, the usage ledger and the Parity Score card.	Code	Report + card

Design principles

- **The golden file is sacred.** Written once, hashed, never editable by any model. Models see failing cases but cannot change expectations.
- **Monotonic repair.** A patch that fixes one behavior but breaks another is rejected.
- **Anti-cheating.** Reject patches that hardcode literals copied from failing inputs. Verify on held-out and adversarial inputs the Debugger never saw.
- **Defined legacy semantics.** The original is compiled with overflow wrapping defined, and this is stated in the report.
- **Honest reporting.** The report lists what was not verified: input counts, seeds, scope limits.

5. Functional requirements

ID	Requirement	Priority
FR-1	Ingest a legacy C module and extract function signatures; validate by successful compile.	Must
FR-2	Generate at least 300 inputs per function: boundary seeds, model-written edge cases, mutations; seeded and deduplicated.	Must
FR-3	Generate and compile a driver for the legacy code with defined overflow behavior.	Must
FR-4	Record original behavior (output and exit status) per input with a timeout into a hashed, immutable golden file.	Must
FR-5	Migrate to Python 3 with identical names and signatures; save the first-pass result as baseline.	Must
FR-6	Replay all inputs on the migrated code with strict comparison (no float tolerance unless configured).	Must
FR-7	Shrink and cluster mismatches; diagnose with a closed taxonomy: rounding, integer overflow, integer division, negative modulo, float formatting, date/calendar, string encoding, other.	Must
FR-8	Repair loop with the no-regression rule, anti-hardcoding check and a six-round cap.	Must
FR-9	Held-out generalization check with fresh seed and fresh model-generated inputs; reported separately.	Must
FR-10	Quirk triage: label suspicious behaviors with evidence; default policy is preserve and flag.	Must
FR-11	Provider-agnostic model client with ordered failover and per-call usage ledger (provider, model, tokens, cost, time).	Must
FR-12	HTML + JSON report and terminal summary (contents in Section 10).	Must
FR-13	Red-team verifier: two rounds of adversarial input generation against the converged migration.	Should
FR-14	Behavior contract: plain-English rules, each linked to evidence inputs.	Should
FR-15	Parity Score card (shareable image or page).	Should
FR-16	Optional adapter to call Meta Model API or Muse Code if available. Not used in the default run.	Could

6. Non-functional requirements

- **Reproducibility:** fixed seeds, golden hash printed in the report, one command reruns the demo.
- **Safety:** legacy code runs in a subprocess with a timeout, no network, temporary directory per run.
- **Speed:** one module end to end in under 5 minutes so the live demo is safe.
- **Cost:** under about $1 per module run; roughly $0.40 estimated at hosted Muse Spark Open prices.
- **Secrets:** API keys live only in an untracked environment file; never committed.
- **Licensing:** every legacy sample is original work; the README carries a "Built with Muse Spark Open" credit line.
- **Resilience:** model responses are cached per run, and a pre-recorded demo is kept as backup.

7. Tech stack

Layer	Choice	Why
Language	Python 3.12	Fast to build; strong subprocess and JSON tooling
Packaging	uv	One-command environment setup on macOS
CLI and terminal UI	Typer and Rich	Clean, readable terminal output for the demo video
Model access	OpenAI-compatible client pointed at partner endpoints	Baseten, Modal and Fireworks all expose compatible APIs; no custom SDK to learn
Data validation	Pydantic	Validates model JSON outputs before use
Boundary fuzzing	Hypothesis plus hand-written boundary tables	Reliable edge cases that do not depend on the model
Legacy runner	clang from Xcode Command Line Tools	Already available on a Mac
Report	Jinja2 templates to a single HTML file	Shareable, easy to screenshot
Tests	pytest	Tests for Parity's own comparison and patch-acceptance logic
Config	Environment file	Provider order, base URLs, keys and model IDs without code changes

The M4 Air is more than enough. Do not try to run Muse Spark Open locally; it needs hundreds of gigabytes of memory. Use partner inference.

8. Models, providers and free credits

Model roles

Role	Model	Served by
All agents (archaeologist, fuzzer, migrator, debugger, red team, quirk triage)	Muse Spark Open	Baseten (primary), Modal, Fireworks
Optional comparison runs	Meta Model API (Muse Spark 1.3) or Muse Code	Meta; only if you choose to add billing. Not used in the default run.

On the Meta credit card

Adding a card to Meta Model API is your decision. A third-party guide says a card is needed because no free tier is confirmed, and the API preview launched for US developers only; neither point is confirmed for your hackathon credits. Parity does not depend on it. If you decide to add one later, use a virtual or low-limit card and set a billing cap.

Credit plan (do in this order, about 10 minutes)

Provider	Action	Reported amount	Role
Baseten	Create an account for the signup credit. Submit the hackathon credit form.	About $30 signup credit (reportedly no card to claim) plus $100 and 2M TPM from the form (reviewed, emailed)	Primary. Use the event-specific model ID from your guide (muse-spark-open-1026 under the meta-models namespace), not the one in Baseten's docs.
Modal	Create a Starter account. Submit the hackathon credit form.	$30 per month Starter credits (reportedly no card) plus $100 from the form for Endpoints	Backup. Unverified whether Starter credits cover Endpoints.
Fireworks	Submit the credit form.	$100 for shortlisted teams only	Optional second backup. Do not rely on it.
Meta Model API	Claim the $150 only if you are comfortable with billing setup.	$150 per your guide	Optional adapter.

Cost estimate

Hosted Muse Spark Open is listed at about $1.25 per million input tokens and $4.25 per million output tokens across providers. A module run of roughly 30 calls costs about $0.40, so $30 covers dozens of runs. Credits are not guaranteed, so cache responses and avoid wasteful reruns.

Open-weights prize hygiene. The partner verifies that the project runs on an eligible Meta open-weight model. Keep the default run on Muse Spark Open only, record the provider and model ID in the usage ledger, keep dashboard screenshots, and state this clearly in the README.

9. Evaluation plan

You write the sample legacy modules yourself, each with a deliberately planted, realistic quirk so you know the ground truth.

Module	Planted behavior	Set
tiered_pricing	Amounts in cents stored in 32-bit integers; overflows on large quantities	Development
invoice_total	Tax rounded half-up via integer math; Python's default rounding differs	Held-out
proration	Day counts across month ends and leap years	Held-out
late_fee (stretch)	Float compounding with truncation to cents	Held-out
currency_convert (stretch)	Integer division truncating toward zero on negatives	Held-out

Tune prompts only on the development module. Held-out modules are never used while tuning; that is your "modules the agent has not seen" claim.

Metrics

Metric	Definition
Behavior Preservation Rate (BPR), first pass	Matching inputs divided by total inputs for the one-shot migration (baseline)
BPR, final	Same measure after Parity's repair loop
Held-out BPR	Match rate on fresh inputs after convergence
Adversarial breaks	Number of equivalence breaks the red team found, and how many were fixed
Quirk detection recall	Planted quirks flagged divided by planted quirks
Iterations, time, cost	Rounds to converge, wall-clock seconds, dollars from the ledger

Report real numbers, including any shortfall. An honest 97% with a clear explanation is more credible than a suspicious 100%. The first-pass versus final comparison is your hero chart.

10. Report and Parity Score card

Section	Content
Headline	Behaviors verified identical (N of M); first-pass vs final vs held-out rates
Agent timeline	Each stage and round: what the agent saw, concluded and changed
Remaining differences	Minimal reproducing inputs for anything still different
Quirk table	Behavior, example inputs, label (likely bug, intentional, unclear), evidence, recommended action
Behavior contract	Plain-English rules with evidence inputs
Not verified	Input counts, seeds, golden file hash, scope limits
Usage ledger	Provider, model, purpose, tokens, cost, time per call
Parity Score card	One-image summary: score, scariest bug found, models used, 'Built with Muse Spark Open'

11. File structure

```
parity/
|-- README.md                what was built during the hackathon, theme, models, provider, credits
|-- LICENSE
|-- pyproject.toml           dependencies (managed by uv)
|-- .gitignore               must list the environment file
|-- .env.example             provider order, base URLs, model IDs (no real keys)
|-- parity/
|   |-- __init__.py
|   |-- cli.py               single entry point: run a module, run the full evaluation
|   |-- llm.py               provider-agnostic client, failover, usage ledger
|   |-- archaeologist.py     signature extraction and probe notes
|   |-- fuzzer.py            boundary seeds, model-written cases, mutation
|   |-- recorder.py          driver build, original execution, golden file + hash
|   |-- migrator.py          C to Python migration, baseline snapshot
|   |-- replayer.py          replay and strict comparison
|   |-- debugger.py          shrink, cluster, diagnose, patch, no-regression check
|   |-- redteam.py           adversarial input generation and re-verification
|   |-- quirks.py            quirk labels and behavior contract
|   |-- report.py            HTML/JSON report and Parity Score card
|   `-- prompts/             one prompt file per agent role
|-- samples/
|   |-- legacy/              your original C modules (3 to 5)
|   `-- expected/            planted-quirk ground truth for scoring
|-- runs/                    per-run output: inputs, golden file, patches, reports
|-- demo/
|   |-- run_demo.sh          one-command demo
|   `-- assets/              score card, screenshots, backup recording
`-- tests/                   tests for comparison, shrinking, patch acceptance
```

12. Prize playbook

Prize	Do this	Avoid this
Best Build on Open Weight Models	Default run on Muse Spark Open only; state provider and model ID in README and report; keep dashboard screenshots; flag the project as open-weight if the platform asks.	Mixing closed Meta API calls into the default run, which muddies the claim.
Best Agentic Use-Case	Show the autonomous loop end to end with no human input; display the agent timeline; quote iterations, success rate, adversarial breaks fixed.	Describing autonomy without showing it.
Most Viral	Post a 30 to 45 second clip plus the score card right after submitting. Lead with the scary bug (an invoice silently turning negative). Tag Meta's AI developer accounts. Submit the post link on the platform. Engagement counts until Nov 8 EOD.	Posting a long technical thread with no visual hook.
Community Vote	Share the gallery link with classmates and developer communities; one vote per verified account.	Anything that looks like vote manipulation.
Indie track	Polished 3-minute video; clean public repo; clear original-work statement.	Unclear boundaries between new and pre-existing work.

13. Execution plan

The original Oct 27 to Nov 4 milestones are now compressed into phases with time budgets. Feature freeze is at T-3 hours; submit at T-90 minutes.

Phase	Budget	Goal	Done when
0 Setup	30 min	Accounts, credits requests, repo, environment, smoke test of the model client.	A one-line model call succeeds through Baseten with no Meta card.
1 Skeleton	2 h	One module end to end: inputs, recording, migration, replay with a printed match rate.	tiered_pricing shows a first-pass rate.
2 Repair loop	2 h	Diagnosis taxonomy, shrinking, no-regression acceptance, round cap.	Match rate rises across rounds on the development module.
3 Evaluation and report	1.5 h	Held-out modules, held-out check, usage ledger, HTML report.	Report shows first-pass vs final vs held-out.
4 Differentiators	1 h (if time)	Red-team verifier, behavior contract, Parity Score card.	Each runs once on a held-out module.
5 Demo and submission	2.5 h (never cut)	Record video, finish README, final push, upload, submit, post on X.	Submission confirmed on the platform.

Scaling by time left

Time left right now	What to do
10 hours or more	Full plan including phase 4; add the fifth module only if everything else is stable.
5 to 9 hours	Phases 0 to 3 plus red team if ahead. Three modules (one development, two held-out).
3 to 4 hours	Phases 0 to 2 on one module, a minimal report, then the demo. Skip held-out claims you cannot support.
Under 3 hours	Submit whatever runs end to end. Be explicit in the README about what is built and what is planned.

Setup checklist (no commands; ask for the exact steps when you start)

- Install the Mac command-line developer tools so clang is available.
- Install uv, create the project, and create the public GitHub repository on day one.

- Create Baseten and Modal accounts; submit the partner credit forms.
- Create the untracked environment file and confirm it is git-ignored before the first commit.
- Run the model-client smoke test; only continue once it succeeds.

14. Three-minute demo video script

Time	On screen	Narration goal
0:00-0:20	Old billing code turning a large invoice into a negative amount	Hook: migrating billing code is scary because you cannot prove nothing changed.
0:20-0:50	Pipeline diagram with agent roles and the model name	Muse Spark Open on partner inference, no human in the loop.
0:50-2:00	Live run on a held-out module: first-pass rate, a mismatch, the diagnosis, the patch, the rerun reaching 100%	Show the agent reasoning, not just the result.
2:00-2:30	Red team finds a new edge case, agent fixes it; report with first-pass vs final vs held-out chart	Evidence, not a claim.
2:30-3:00	Quirk table, behavior contract, usage ledger, Parity Score card, repo link	What was built during the hackathon; invite the viewer to the repo.

Recording tips: use the built-in Mac screen recorder, large terminal font, a clean desktop, and keep a pre-recorded backup of the live run in case the provider is slow. Upload as unlisted and open the link in a private window to confirm it plays.

15. Submission checklist

Done	Item
[ ]	Repository is public (open it in a private window to confirm)
[ ]	README states what was built during the hackathon, the theme, the models, the provider and exact model ID
[ ]	README and report include the line "Built with Muse Spark Open"
[ ]	No API keys anywhere in the repo or its history
[ ]	Demo video is 3 minutes and plays when logged out
[ ]	Everyone who contributed is listed on the submission page
[ ]	Track selected: Indie Developer
[ ]	X post published with clip and score card; post link submitted on the platform
[ ]	Gallery link shared for the Community Vote
[ ]	Submitted at least 90 minutes before 9:00 PM EST

16. Risk register

Risk	Likelihood	Mitigation
Not enough time left today	High	Use the scaling table; submit a smaller working version rather than a bigger broken one.
Partner credits do not arrive in time	High	Use signup credits (Baseten, Modal); cache responses; keep runs small; Meta adapter as last resort.
Provider outage or rate limit during the demo	Medium	Automatic failover across providers; pre-recorded backup run.
Model fits the tests instead of fixing the logic	Medium	No-regression rule, anti-hardcoding check, held-out and adversarial verification.
Model output is not valid JSON or includes reasoning text	Medium	Strict parsing and validation with one automatic retry.
Overclaiming 'proves'	Medium	Use 'verified on N recorded behaviors' everywhere.
Seen as a generic code translator	Medium	Lead the demo with the evidence and the bug report, not the translation.
Open-weight prize claim muddied by closed API calls	Low	Default run uses Muse Spark Open only.
Legacy samples with nondeterminism	Low	Samples are pure by design; state this limit in the report.

17. Definition of done

Parity is submittable when one command takes a legacy module from first-pass mismatches to a final match rate, prints a held-out result, and writes an HTML report with a usage ledger, in under five minutes, from a public repository that has a README and a three-minute video. Anything beyond that (red team, behavior contract, score card) raises your odds but is not required to submit.

Appendix: verified facts and assumptions

Statement	Status
Deadline is Nov 4, 9:00 PM Eastern; building period Oct 27 to Nov 4; judging Nov 5 to 8; closing ceremony Nov 9	From your Participant Guide and the event page
Muse Spark Open weights are available via Baseten, Fireworks, Modal and other hosts	Meta announcement (Oct 2026)
Use the event-specific Baseten model ID from your guide	From your Participant Guide
Hosted Muse Spark Open pricing about $1.25 in / $4.25 out per million tokens	Third-party price listings; verify on the provider page
Baseten about $30 signup credit, no card to claim; Modal Starter $30 per month, no card	Third-party reports; verify on provider sites
Meta Model API requires a credit card	Unconfirmed; one third-party guide says so
Muse Code supports non-interactive use	Unverified; not on the critical path
The license asks derivatives to display 'Built with Muse Spark Open'	Reported by one third-party source; a README credit line costs nothing
No PDFs were attached to the chat, so the official rules PDF was not reviewed	Re-check the official rules you accepted for anything that conflicts